

Проект 13.17 — Автономный дрон без GPS

Навигация без GPS на основе компьютерного зрения и VINS-Mono.

- Боевой борт: NVIDIA Jetson Orin Nano (ARM64, JetPack 6 / L4T r36).
- Симуляция: ноутбук x86 + NVIDIA GPU (ArduPilot SITL + Gazebo Harmonic).

Структура репозитория

```
docker/  
  orin/      — БОЕВОЙ стек (Jetson, реальная камера + полётник)  
  sim/      — СИМУЛЯЦИЯ (x86 + NVIDIA: SITL + Gazebo + VINS)  
src/  
  camera/   — C++ CUDA камера-нода (camera_pkg) + tuner  
  vins/     — VINS-MONO-ROS2 (конфиги) + python cam-ноды (fallback)  
  sim/      — симуляционная обвязка (байеризатор, launch)  
  nav/      — пакет nav_pkg: нейросети навигации (NN1/NN2, пока болванки)  
              + openhd_streamer (даунлинк в OpenHD с оверлеем детекций)  
  orin_shutdown/ — Go-утилита graceful shutdown через MAVLink  
  concept.txt — исходная концепция проекта  
distro/     — деплой на Orin (etc/, home/andriy/, usr/) — systemd, сети, скрипты  
tools/mdtopdf/ — генератор CLAUDE.pdf (reportlab)
```

Бинарники в репе — пустые заглушки *__bin (реальные собираются на месте).

Навигация (два слоя)

Нейросеть №1 — якорная локализация (замена GPS, абсолютная точность):

YOLOv8 / SuperPoint+LightGlue находит известные ориентиры → Ray Tracing через
intrinsics + барометр/IMU → абсолютная позиция → сбрасывает дрейф VINS-Mono.

Нейросеть №2 — топологическая карта (семантика): DINOv2 + AnyLoc + FAISS,
сжимает сцену в дескриптор, сравнивает с базой облёта → управление «по смыслу».

Нейросеть №2 ведёт дрон, Нейросеть №1 периодически сбрасывает дрейф.

Реализация — пакет nav_pkg (src/nav/, ament_python). См. «OpenHD-оверлей».

Камера и C++ нода

src/camera/camera_node.cpp (camera_pkg) — боевая ROS2 нода:

- V4L2 захват (v4l2-ctl, формат BA10) → CUDA дебайер (cv::cuda::demosaicing,
BayerGB2RGB) → публикует mono8 в /image_mono (вход VINS)
- публикует /image_color (bgr8, полный кадр, каждый кадр) — вход nav-стороны
(нейросети + openhd_streamer). Цвет приводится к настоящему BGR
(cv::cuda::cvtColor RGB→BGR); моно-путь VINS считается ДО конверсии (не изменён)

- ROS-параметры на лету: `gain/r/g/b`; `device` (путь к V4L2, default `/dev/video0`) — ключ к работе в симуляции (см. ниже)
 - `stream_opendhd` (default `false`) — встроенный энкодер OpenHD :5600.
- По умолчанию ВЫКЛ: поток теперь собирает `opendhd_streamer` (рисует рамки NN).
- `true` — standalone-режим камеры без nav-стороны
- штампует кадр через `get_clock() -> now() → уважает use_sim_time`

`src/camera/tuner/cuda/main.cpp` — автономный веб-тюнер (без ROS): :8080 GUI, :5000 mono MJPEG, :5001 color MJPEG. Запускается вручную.

Разные коды Байера в CUDA (BayerGB2RGB) и CPU (BayerGR2BGR) — норма: в OpenCV CUDA-модуле сдвиг в именовании паттернов. Оба обрабатывают один физический паттерн (GRBG сенсора).

OpenHD-оверлей и nav_pkg (вариант 2)

Видео для оператора (OpenHD) проходит через нейросети, которые рисуют рамки вокруг объектов. Нейросети работают РЕДКО (NN1 ~1 Гц, NN2 ~3 с) и НЕ должны гейтить fps видео. Поэтому энкодер вынесен в отдельную ноду, а нейросети шлют только геометрию/семантику (килобайты, не пиксели):

```

camera_node → /image_color ──┐
                             └─┬─> nn1_anchor (1Гц) → /nn1/detections (Detection2DArray)
                             └─┬─> nn2_scene (3с) → /nn2/scene (String, метка)
                             └─┬─> opendhd_streamer ← кэш последних детекций
                             └─┬─> рисует оверлей на КАЖДОМ кадре → H.264 → OpenHD :5600

```

`src/nav/` = пакет `nav_pkg` (ament_python):

- `opendhd_streamer` — подписан на `/image_color + /nn1/detections + /nn2/scene`, кэширует последние детекции, рисует на каждом кадре (`cv2.rectangle/putText`), ужимает до 640×360, кодирует H.264 (GStreamer → `udpsink :5600`). Видео на полном fps независимо от инференса; рамки «залипают» между обновлениями (для FPV норм).

- `nn1_anchor` — Нейросеть №1 (якорная локализация). Инкремент 1: SuperPoint+ LightGlue (`anchor_matcher.py`) матчит `/image_color` против географической базы облёта (`data/reference_db/`) → `bbox+id` ориентирует в `/nn1/detections`.

- `ray_tracer` — Инкремент 2: засечка по ориентиру (`geo.py`: луч через `intrinsics` + углы MAVROS + баро → абсолютная позиция в ENU) → поправка-смещение к VINS (сброс дрейфа) → `/nn1/anchor_pose`, `/nn1/corrected_odom`, `/nn1/drift`.
- Инкремент 3: публикует скорректированную позу в `/mavros/vision_pose/pose` (ArduPilot EK3 External Nav) — `ray_tracer` = единственный мост VINS → полётник.
- Осталось: yaw-коррекция + FAISS-префильтр. На FCU нужен `EK3_SRC1_POSXY=6`.
- Детали и допущения: `src/nav/tools/nn1_anchor_howto.txt`.

- `nn2_scene` — пока БОЛВАНКА: таймер 3 с, последний кадр, фиктивная метка.
- Запуск: `ros2 launch nav_pkg nav.launch.py use_sim_time:=true` (камеру/VINS)

не поднимает). В симуляции включается из `sim_nav.launch.py` (`IncludeLaunch`).

Зависимость `vision_msgs` (`Detection2DArray`) — добавлена в образ `nav` (в `ros-base` её нет). Вариант 2 выбран ради чистоты архитектуры; межнодовый `republish` полного `/image_color` — осознанная плата, потом можно оптимизировать.

Боевой борт (`docker/orin/`): камера больше не гонит OpenHD сама (`stream_openhd:=false`) — `openhd_streamer` запускается рядом с камерой в `vins_service.sh/vins_service_m.sh` (PID4). На Orin поднимается ТОЛЬКО стример (не болванки NN1/NN2): отдаёт чистое видео, рамки появятся, когда боевые нейросети начнут публиковать `/nn1/detections`, `/nn2/scene.src/nav` смонтирован в контейнер, в `Dockerfile` добавлен `vision_msgs`.

VINS-Mono

Работает при армировании дрона. Конфиги: `src/vins/VINS-MONO-ROS2/config_pkg/config/`

- `dummy_13_7.yaml` — боевой (реальный ArduCam)
- `sim.yaml` — симуляция (см. ниже)

Боевой стек — `docker/orin/`

База: `dustynv/ros:humble-ros-base-l4t-r36.3.0` (NVIDIA L4T: ROS2 Humble + CUDA + OpenCV-c-CUDA даром). `runtime: nvidia, network_mode: host`, пропуск `/dev/video0`. Контейнер `vins_project_13_7`. Сборка через `colcon` внутри контейнера (монтируются `src/vins`, `src/camera`, `src/nav`).

Systemd (в `distro/etc/systemd/system/`): `mavros`, `vins/vins_m` (запуск VINS; суффикс `_m` = ручной режим без ожидания арминга), `auto-bag/auto-bag-m`, `orin-shutdown`. Скрипты — `distro/home/andriy/`.

Симуляция — `docker/sim/`

Запуск всей системы на ноутбуке БЕЗ железа. Три контейнера:

| Сервис | База | GPU | Роль |

|---|---|---|---|

| `simulator` | `osrf/ros:humble-desktop + Gazebo Harmonic` | `graphics+compute` | `Gazebo + ArduPilot SITL + ros_gz_bridge` |

| `mavlink_router` | `radarku/mavlink-router` | — | раздача MAVLink по UDP |

Ключевые решения по симуляции

1. Gazebo Harmonic (не Fortress) — по concept.txt (плагины gz-sim-*).

Нештатная пара для Humble, ставится вручную из репозитория osrfoundation.

2. GPU = NVIDIA CUDA (езде).

3. Вариант А — OpenCV+CUDA из исходников в образе nav. Камера-нода работает в симуляции «как есть» (с CUDA-дебайером). База nav — devel CUDA-образ.

□□ apt'шный cv_bridge тянет системный OpenCV (4.5.4 без CUDA) → ABI-краш.

Поэтому cv_bridge собирается из исходников против нашего OpenCV (overlay /opt/overlay). CUDA_ARCH_BIN — build-arg под GPU нота.

4. Камера через v4l2loopback (камера-нода НЕ переписывается):

、

Gazebo /camera/image_raw (RGB)

→ bayerizer.py (RGB → Bayer16)

→ write() → /dev/rawbayer (v4l2loopback, модуль ядра ХОСТА)

→ camera_node (device:=/dev/rawbayer) → CUDA дебайер → /image_mono

、

Меняется только параметр device. Байеризатор: src/sim/bayerizer.py

(паттерн GRBG по умолчанию). v4l2loopback ставится на хосте (modprobe, симлинк /dev/rawbayer), пробрасывается в nav через devices:.

5. isaac_ros удалён — драйвер камеры не поддерживает Argus.

6. Разрешение 1280×720 — под него захардкожена camera_node и посчитан sim.yaml. Камеру дрона в Gazebo держать 1280×720 (не 1920×1200).

7. mavlink_router — клиент к SITL (tcp:5760). В concept.txt был конфликт портов (два сервера на 5760) — исправлено.

8. use_sim_time всем нодам через src/sim/sim_nav.launch.py. Исключение — ros_gz_bridge (он источник /clock). MAVROS — отдельно (тонкий момент: часть штампов IMU от FCU).

Мир — Military fortress

docker/sim/worlds/mili_fortress.sdf — карта из

[engcang/gazebo_maps](https://github.com/engcang/gazebo_maps) (mili_tech),

на которой в демо-видео доказанно работает VINS-Fusion+YOLO. Выбрана потому, что текстуры уже подходят для VINS (свой мир в Blender легко «промахнуться»).

Ассеты вендорены в worlds/mili_tech/ (~27 МБ, CC; ATTRIBUTION.md).

Портирование Classic → Harmonic: добавлены обязательные system-плагины,

<population> → явная расстановка, <road> убран, ode → DART, Ogre-материалы
(grass_plane/digital_wall) → PBR, убрана битая ссылка model://home.

Дрон — worlds/iris_cam/

iris_with_ardupilot из ardupilot_gazebo (проверена под SITL: моторы, IMU, ArduPilotPlugin@9002) + камера пилота (параметры из concept.txt: поза 0.15 0 0.05, наклон 0.26, fov 1.5708; но 1280×720). Прикреплена к iris_with_standoffs::base_link, публикует gz-топик camera/image_raw.

sim.yaml (VINS для Gazebo)

Отличия от боевого: нулевая дисторсия (идеальный pinhole), интринсики $f_x=f_y=640$, $c_x=640$, $c_y=360$ (fov 90° @ 1280×720), пути /root/sim_ws/, экстринсики из позы камеры в SDF, заниженный шум IMU (нет вибраций рамы).

Полный пайплайн

`gz sim (мир+дрон) → SITL ↔ ArduPilotPlugin → ros_gz_bridge (camera + /clock)
→ bayerizer → /dev/rawbayer → camera_node → /image_mono → VINS;
MAVROS ↔ mavlink_router ↔ SITL. Команды — в docker/sim/README.md`.

☐☐ Не протестировано без Gazebo (проверить на ноуте)

- Вложенные <include> в mili_map (если карта не появится — расплющить)
- mt_background (heightmap) — может ломать загрузку
- Scope iris_with_standoffs::base_link (крепление камеры)
- Соединение SITL (порт 9002), синхронизация IMU/камеры под use_sim_time
- Формат BA10 на v4l2loopback (нода берёт sizeimage = wh2)

Репозиторий

- GitHub: <https://github.com/linux100talion/13.17>, ветка main
- Git user: Andriy Kutsevol andriykutsevol@gmail.com
- CLAUDE.pdf генерится: /tmp/pdfenv/bin/python3 tools/mdtopdf/claudepdf.py